CAHIER DES CHARGES

Projet : Agent IA local

Node.js / TypeScript · API REST · Ollama · RAG · SQLite · Outils

Document destiné aux étudiants — Travail à réaliser en autonomie

Ce document décrit le besoin, les contraintes et l'architecture attendue. Il ne fournit volontairement aucune solution d'implémentation (code, librairies imposées au détail, recettes pas à pas). Les choix techniques fins et la réalisation sont à la charge de l'équipe étudiante.

1. Contexte et objectifs

1.1 Contexte

Les modèles de langage locaux (via Ollama) permettent de construire des assistants capables de raisonner, d'utiliser des outils et de s'appuyer sur une base documentaire. Ce projet vise à concevoir et développer un tel agent de A à Z, exposé derrière une API HTTP.

1.2 Objectifs pédagogiques

- Comprendre l'architecture d'un agent IA (LLM + outils + mémoire + RAG).
- Mettre en place une API REST professionnelle en TypeScript.
- Intégrer un modèle local Ollama (chat et embeddings).
- Persister des données applicatives et conversationnelles.
- Indexer et interroger des documents (RAG).
- Appliquer des règles de sécurité minimales sur les outils.

1.3 Objectif produit

Livrer un service capable de recevoir une question utilisateur, d'enrichir éventuellement le contexte via des documents indexés, d'appeler des outils si nécessaire, puis de renvoyer une réponse structurée.

2. Périmètre

2.1 Dans le périmètre (MVP)

- API REST (pas d'interface web obligatoire).
- Connexion à un serveur Ollama unique hébergeant au moins un modèle de chat et un modèle d'embeddings.
- Base de données SQLite locale.
- Sessions de conversation persistées.
- Pipeline RAG sur fichiers texte placés dans un dossier dédié.
- Système d'outils (tools) appelables par l'agent.
- Documentation de démarrage et procédures de développement.

2.2 Hors périmètre

- Interface utilisateur web / mobile.
- Authentification multi-utilisateurs, JWT, OAuth.
- PostgreSQL, Redis, files de messages.

- Déploiement cloud / Kubernetes.
- Agents multi-agents complexes (graphes d'agents).
- Streaming temps réel (SSE/WebSocket) — optionnel si le temps le permet.

3. Architecture globale

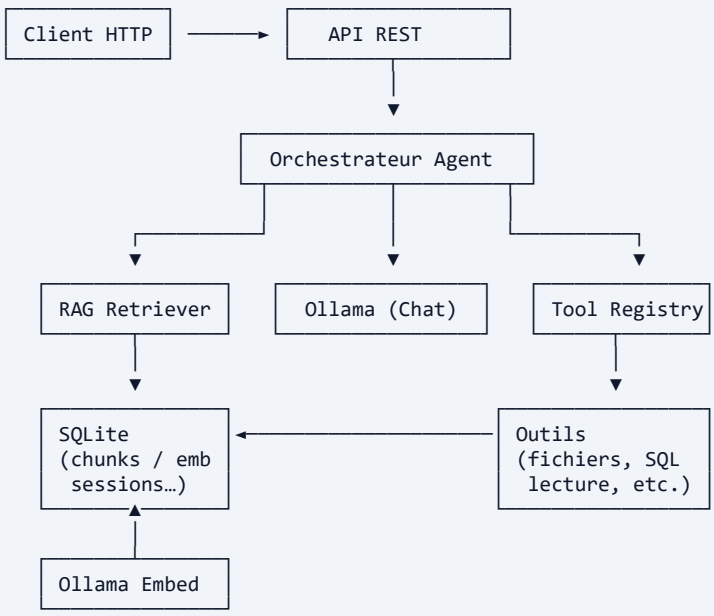
3.1 Vue d'ensemble

Le système est composé de plusieurs blocs logiques. L'étudiant doit respecter cette décomposition conceptuelle, sans que l'organisation exacte des fichiers ou le choix des librairies internes ne soient imposés.

- Client HTTP : outil de test (curl, Insomnia, script, etc.).
- API REST : point d'entrée unique du service.
- Orchestrateur agent : coordonne prompt, RAG, appels LLM et outils.
- Client LLM (Ollama) : chat + embeddings.
- Registre d'outils : déclaration et exécution contrôlée des tools.
- Couche données : SQLite (sessions, messages, données métier, index RAG).
- Corpus documentaire : fichiers sources du RAG.

3.2 Schéma d'architecture

Schéma logique du flux principal :



Note : un seul serveur Ollama ; deux usages (chat / embeddings).

3.3 Boucle de traitement d'une requête chat

Séquence attendue (comportement), sans imposer l'implémentation :

- 1. Réception d'un message (+ identifiant de session optionnel).
- 2. Recherche éventuelle de passages documentaires pertinents (RAG).
- 3. Construction du contexte envoyé au modèle (consignes + historique + extraits).
- 4. Appel au modèle de chat ; si le modèle demande un outil, l'exécuter puis relancer.
- 5. Bornage du nombre d'itérations outils.
- 6. Persistance des messages et retour d'une réponse structurée (texte, outils utilisés, sources).

3.4 Principes d’architecture à respecter

- Séparation des responsabilités (API ≠ orchestrateur ≠ RAG ≠ outils ≠ BDD).
- Configuration externalisée (variables d’environnement).
- Un seul host Ollama pour chat et embeddings.
- Pas d’accès filesystem ou SQL non contrôlé depuis les tools.

4. Exigences fonctionnelles

4.1 Santé du service

Le service doit exposer un endpoint de santé indiquant au minimum :

- Disponibilité de la base de données.
- Disponibilité d’Ollama.
- Un indicateur global de bon fonctionnement.

4.2 Conversation (chat)

- Accepter un message utilisateur.
- Gérer une session (création automatique si absente, reprise si fournie).
- Conserver un historique de conversation exploitable par l’agent.
- Retourner : identifiant de session, réponse textuelle, liste des outils utilisés, sources RAG éventuelles.
- Permettre la suppression d’une session.

4.3 Outils (tools)

L’agent doit pouvoir utiliser au minimum les capacités suivantes :

- Lister les documents disponibles du corpus.
- Lire le contenu d’un fichier du corpus (accès restreint).
- Exécuter une requête SQL en lecture seule sur des tables autorisées.
- Évaluer une expression arithmétique simple (outil de démonstration).

Les outils doivent être déclaratifs (nom, description, schéma de paramètres) afin que le modèle puisse les choisir.

4.4 RAG (Retrieval-Augmented Generation)

- Indexer un fichier ou un dossier du corpus.
- Découper les documents en chunks.
- Produire et stocker des embeddings via Ollama.
- Retrouver les passages les plus pertinents pour une question.
- Injecter ces passages dans le contexte avant la génération.
- Lister les documents déjà indexés.

Formats minimum attendus pour le corpus : Markdown, TXT, JSON, CSV.

4.5 Données métier de démonstration

- Prévoir au moins une table métier simple (ex. contacts ou produits) pour illustrer l’outil SQL.
- Fournir un moyen de peupler ces données de démo (script de seed).

5. Contrats d’API attendus

Les chemins exacts peuvent varier, mais le service doit couvrir ces capacités. Proposition de contrat cible :

Méthode	Route	Rôle
GET	/health	État Ollama + base
POST	/chat	Message agent (session optionnelle)
POST	/ingest	Indexation RAG d'un chemin
GET	/documents	Liste des documents indexés
DELETE	/sessions/:id	Suppression de session

Le détail des corps JSON (validation, codes HTTP d'erreur) est à concevoir par l'équipe, de façon cohérente et documentée.

6. Contraintes techniques

6.1 Stack imposée

- Runtime : Node.js 20 ou supérieur.
- Langage : TypeScript (mode strict recommandé).
- Interface : API REST.
- LLM : Ollama (instance locale unique).
- Base : SQLite (fichier local).
- RAG : embeddings locaux via Ollama + similarité côté application ou SQLite.

6.2 Configuration

- Port d'écoute configurable.
- URL Ollama configurable.
- Noms des modèles chat et embeddings configurables.
- Chemins base de données et corpus documentaire configurables.
- Limites : timeout LLM, nombre max d'itérations outils, top-k RAG, taille/overlap des chunks.

6.3 Qualité logicielle

- Projet initialisable clairement (README de démarrage ≤ 10 minutes).
- Tests automatisés sur au moins les briques critiques (ex. chunking, similarité, outils).
- Contrôle de types TypeScript sans erreur.

7. Exigences non fonctionnelles

- Sécurité : pas d'exécution shell arbitraire ; lecture fichiers limitée au corpus ; SQL lecture seule + allowlist de tables.
- Robustesse : gestion des erreurs Ollama / BDD / outils avec messages clairs.
- Observabilité minimale : logs applicatifs sur les requêtes et erreurs.
- Maintenabilité : code lisible, modules séparés, configuration claire.
- Reproductibilité : instructions pour installer Ollama, tirer les modèles, migrer la BDD, lancer le service.

8. Livrables attendus

- Code source du service (TypeScript).
- Fichier d'exemple de configuration (sans secrets).

- Script(s) d'initialisation de la base et de données de démo.
- Corpus documentaire d'exemple.
- README de démarrage.
- Procédures de développement (setup, ajout d'un outil, pipeline RAG, API/tests, troubleshooting).
- Présentations (architecture, guide développeur, procédures) — format libre (PPTX recommandé).
- Jeux de tests exécutables.

9. Scénarios de validation

Le projet sera considéré fonctionnel si les scénarios suivants passent :

- S1 — Health : le service indique Ollama et BDD disponibles lorsque l'environnement est correct.
- S2 — Chat simple : une question générale obtient une réponse textuelle et un sessionId.
- S3 — Outil calcul : une question arithmétique déclenche l'outil dédié.
- S4 — Outil fichiers : l'agent liste / lit un document du corpus.
- S5 — Outil SQL : l'agent interroge la table métier en SELECT.
- S6 — RAG : après ingest, une question sur le contenu documentaire s'appuie sur des sources.
- S7 — Sécurité : une tentative de chemin hors corpus ou de SQL non SELECT est refusée.
- S8 — Session : reprise d'historique avec le même sessionId.

10. Critères d'évaluation

Critère	Poids indicatif	Attendu
Architecture & séparation des modules	20 %	Blocs clairs, schéma respecté
API & agent (chat + tools)	25 %	Boucle tools, sessions, réponses structurées
RAG	20 %	Ingest, retrieval, sources
Sécurité & robustesse	15 %	Allowlists, erreurs gérées
Qualité (tests, types, README)	10 %	Démarrage reproductible
Documentation / procédures / supports	10 %	Clairs et cohérents avec le code

11. Consignes de travail

- Travailler en équipe (taille définie par l'enseignant).
- Ne pas coller une solution toute faite sans la comprendre : l'oral de soutenance portera sur vos choix.
- Documenter vos décisions d'architecture (courte note dans le README ou un ADRs léger).
- Prévoir une démo : health → ingest → chat avec outils et sources.

Rappel : ce cahier des charges fixe le « quoi » et l'architecture globale. Le « comment » (librairies précises, structure de dossiers, algorithmes exacts) est à inventer et justifier.

— Fin du cahier des charges —